MACHINE LEARNING

UNIT – II : Nearest Neighbor-Based Models

Easy digital notes with diagrams for students (JNTU B.Tech, 23A3140IT)

Topics in this unit

1. Introduction to proximity measures and distance measures
2. Non-metric similarity functions
3. Proximity between binary patterns
4. Classification algorithms based on distance measures
5. K-Nearest Neighbor (KNN) classifier and choosing k
6. Radius distance nearest neighbor algorithm
7. KNN regression
8. Performance of classifiers
9. Performance of regression algorithms

> * **Study tip:**
> Do one numeric example of each measure by hand - exam questions are mostly 'calculate the distance / classify the point / find accuracy'. The worked examples here use small tables so you can copy the method.

Syllabus (as given)

Nearest Neighbor-Based Models: Introduction to Proximity Measures, Distance Measures, Non-Metric Similarity Functions, Proximity Between Binary Patterns, Different Classification Algorithms Based on the Distance Measures, K-Nearest Neighbor Classifier, Radius Distance Nearest Neighbor Algorithm, KNN Regression, Performance of Classifiers, Performance of Regression Algorithms.

Part I : Proximity and Distance Measures

2.1 Introduction to Proximity Measures

Proximity tells how **close or alike** two patterns are. It has two faces: **similarity** (large value = more alike) and **dissimilarity / distance** (small value = more alike). Nearest-neighbor models are built on one idea: **similar inputs give similar outputs**.

- **Proximity matrix:** for n patterns, an n × n table whose (i, j) entry is the distance (or similarity) between pattern i and j. It is symmetric with zeros on the diagonal for distances.
- **Instance-based / lazy learning:** KNN does **no training**. It stores all examples and does the work only when a query arrives (memory-based). Compare with **eager** learners (tree, SVM) that build a model first.

- **Why choice of measure matters:** the neighbors found, and hence the answer, change with the measure. Numeric data: Euclidean, Manhattan. Binary data: Hamming, Jaccard. Text: cosine. Probabilities: KL divergence.
- **Conversion:** a similarity s in $[0,1]$ gives a distance $d = 1 - s$; a distance gives similarity $s = 1 / (1 + d)$.

2.2 Distance Measures

A function $d(x, y)$ is a **metric (true distance)** if it satisfies four rules for all patterns x, y, z:

Property	Rule	Meaning
Non-negativity	$d(x, y) \geq 0$	distance is never negative
Identity	$d(x, y) = 0 \Leftrightarrow x = y$	zero only for the same point
Symmetry	$d(x, y) = d(y, x)$	A to B equals B to A
Triangle inequality	$d(x, z) \leq d(x, y) + d(y, z)$	a detour is never shorter

For d-dimensional vectors $x = (x1..xd)$, $y = (y1..yd)$ the common distances are:

Measure	Formula	Remark
Euclidean (L2)	$d = \text{sqrt}(\Sigma (xi - yi)^2)$	straight-line; most used; sensitive to scale
Manhattan (L1)	$d = \Sigma \lvert xi - yi \rvert$	city-block; less affected by outliers
Chebyshev (L∞)	$d = \max \lvert xi - yi \rvert$	largest single difference
Minkowski (Lp)	$d = (\Sigma \lvert xi - yi \rvert ^ p)^{(1/p)}$	general form; p = 1, 2, ∞ give the three above
Mahalanobis	$d = \text{sqrt}((x - y)^T \Sigma^{-1} (x - y))$	uses covariance Σ; removes scale and correlation effects

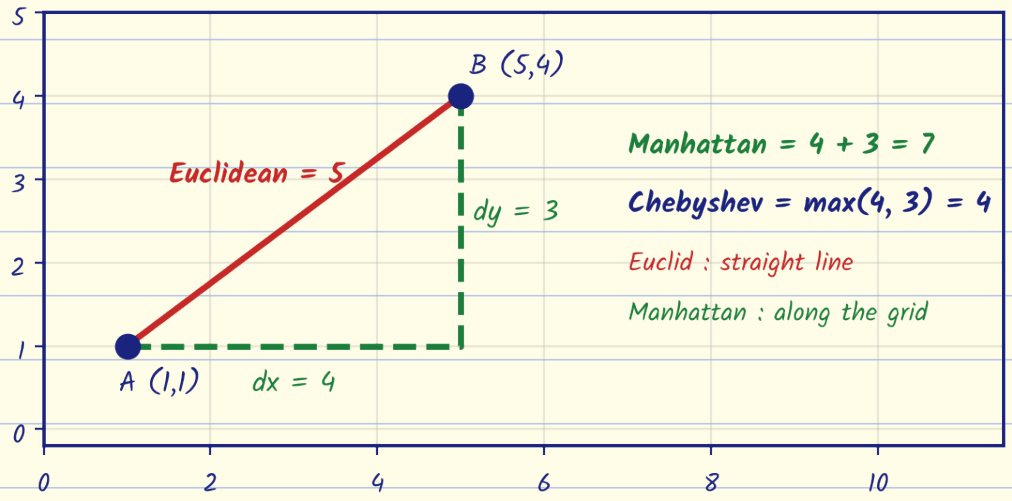


Fig 2.1 : Euclidean, Manhattan and Chebyshev paths between two points

- **Worked example:** $x = (1,1)$, $y = (5,4)$. Euclidean = $\text{sqrt}(4^2 + 3^2) = \mathbf{5}$. Manhattan = $4 + 3 = \mathbf{7}$. Chebyshev = $\max(4,3) = \mathbf{4}$. Minkowski $p = 3$ = $(4^3 + 3^3)^{(1/3)} = (91)^{(1/3)} = \mathbf{4.50}$.
- Always **Chebyshev ≤ Euclidean ≤ Manhattan**. Larger p gives more weight to the biggest difference.

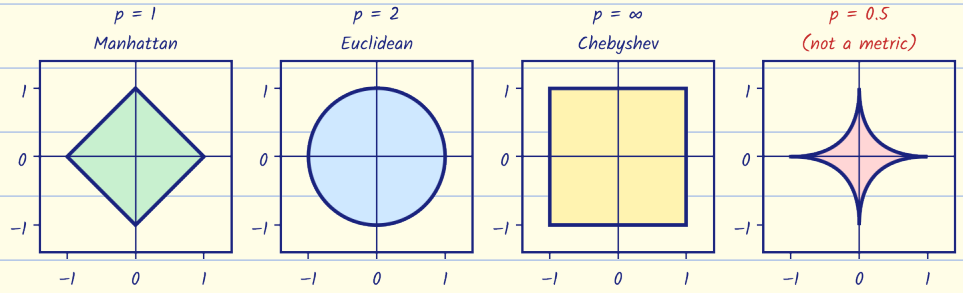


Fig 2.2 : Unit balls (points at distance 1 from origin) for different p

- **Mahalanobis** example: mean 0, Σ = [[4,0],[0,1]]. For x = (2,0): $d^2 = 2^2/4 = 1$, so d = 1. For y = (0,2): $d^2 = 2^2/1 = 4$, so d = 2. Both have Euclidean distance 2, but y is further from the data cloud because feature 2 varies less. If Σ = I, Mahalanobis = Euclidean.

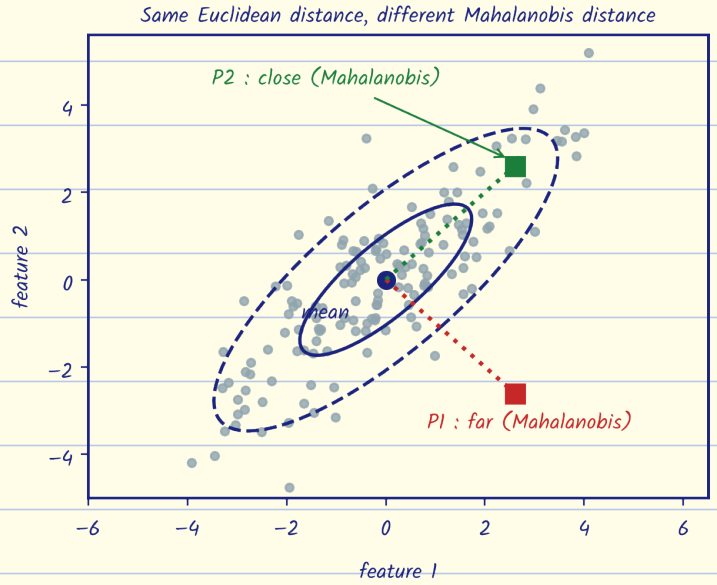


Fig 2.3 : Mahalanobis distance follows the shape of the data

> * **Scaling matters:**
> A feature with a large range (salary) can dominate Euclidean distance. **Always scale** features first: min-max $x' = (x - min)/(max - min)$ or z-score $z = (x - \mu)/\sigma$.

Person	Age	Salary (Rs)
q	30	50,000
A	60	50,100
B	31	60,000

Unscaled: d(q,A) = 104.4

d(q,B) = 10,000.0

Salary swamps age: A looks 'nearest' although 30 yrs older

After min-max scaling (0 to 1):

range age 30-60, salary 50k-60k

q = (0, 0)

A = (1, 0.01) d = 1.00

B = (0.03, 1) d = 1.00

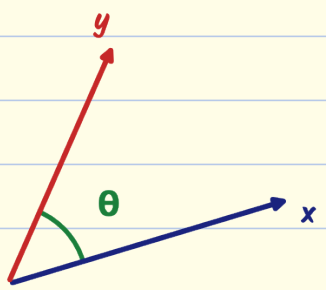
Now both features count equally, **so scale BEFORE using k-NN.**

Fig 2.4 : Without scaling the large-range feature decides the neighbor

2.3 Non-Metric Similarity Functions

Many useful measures **break one or more metric rules** (usually the triangle inequality or the identity rule). They are called **non-metric**. They still work well for matching, but tricks that need a true metric (e.g. some fast search trees) may not apply.

Function	Formula	Why it is non-metric
Cosine similarity	cos θ = x·y / (|x| |y|)	a similarity; 1 - cos θ fails the triangle inequality; x and 2x give 1 (identity fails)
KL divergence	KL(P||Q) = Σ P(i) ln(P(i)/Q(i))	not symmetric: KL(P||Q) ≠ KL(Q||P)
Squared Euclidean	Σ (xi - yi)²	triangle inequality fails (see figure)
Pearson correlation	r = cov(x,y) / (σx σy)	similarity in [-1, 1]; 1 - r not a metric
Fractional Lp, p < 1	(Σ |xi - yi|^p)^(1/p)	unit ball is not convex; triangle fails
Dynamic time warping	best alignment cost of two series	triangle inequality can fail



cos θ = (x·y) / (|x| |y|)

θ small → cos θ ≈ 1

θ = 90° → cos θ = 0

Only direction matters, not length.

Fig 2.5 : Cosine similarity measures the angle, not the length

- **Cosine example:** x = (1,2,0), y = (2,1,1). x·y = 2 + 2 + 0 = 4; |x| = sqrt5 = 2.236; |y| = sqrt6 = 2.449. cos θ = 4 / 5.477 = **0.730**. Cosine distance = 1 - 0.730 = **0.270**. Used for **text documents** (word-count vectors), where length is irrelevant.
- **KL example:** P = (0.5, 0.3, 0.2), Q = (0.8, 0.1, 0.1). KL(P||Q) = **0.233** but KL(Q||P) = **0.197**. Different, so it is **not symmetric**. Symmetric fix: Jensen-Shannon divergence.

4 > 1 + 1 : triangle inequality FAILS

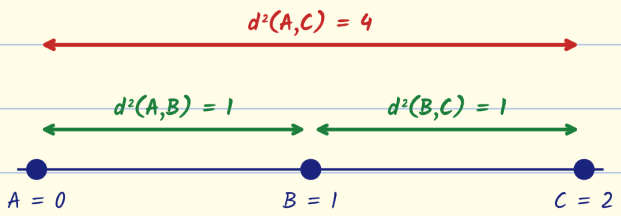


Fig 2.6 : Squared Euclidean distance violates the triangle inequality

2.4 Proximity Between Binary Patterns

For patterns of 0/1 features (present / absent) we count matches and mismatches. For two binary vectors X, Y of length n form the **2 x 2 contingency table**:

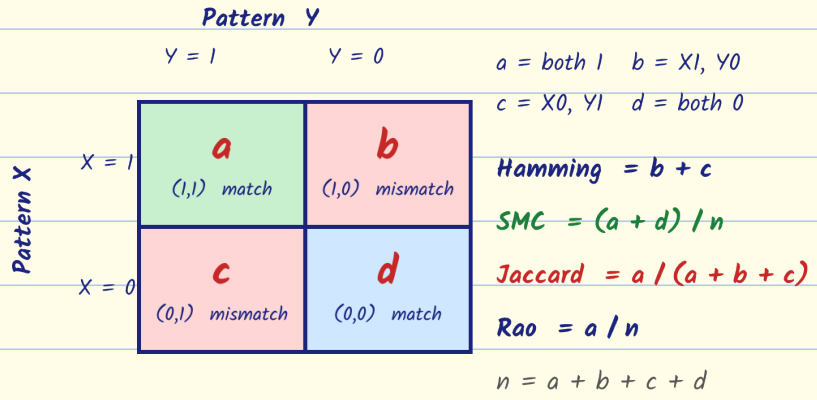


Fig 2.7 : Contingency table for binary patterns X and Y

- **a** = positions where both are 1, **b** = X is 1, Y is 0, **c** = X is 0, Y is 1, **d** = both 0. n = a + b + c + d.

Measure	Formula	Use
Hamming distance	b + c	number of differing bits
Simple Matching Coefficient (SMC)	(a + d) / n	symmetric binary data (0 and 1 equally important)
Jaccard coefficient	a / (a + b + c)	asymmetric data: 0-0 matches ignored (e.g. items bought)
Rao coefficient	a / n	counts only common 1s, relative to all positions
Jaccard distance	1 - a/(a + b + c)	a metric

- **Worked example:** X = 1 1 0 1 0 0 1 0 and Y = 1 0 0 1 1 0 0 0. Counting: a = 2, b = 2, c = 1, d = 3, n = 8.
- Hamming = b + c = **3**. SMC = (a + d)/n = 5/8 = **0.625**. Jaccard = a/(a+b+c) = 2/5 = **0.400**. Rao = a/n = 2/8 = **0.250**.
- **SMC vs Jaccard:** with many zeros (sparse data) SMC is large just because of 0-0 matches; Jaccard avoids this.

Part II : Classification Using Distance

2.5 Classification Algorithms Based on Distance Measures

These classifiers label a new pattern using its **distance to training patterns**. No equation of a boundary is learned.

Algorithm	Idea	Note
Nearest Neighbor (NN, 1-NN)	give q the class of the single closest training pattern	simple; sensitive to noise; error ≤ 2 × Bayes error (Cover-Hart)
K-Nearest Neighbor (KNN)	majority vote of the k closest	k reduces noise effect
Weighted KNN	closer neighbors get larger vote, $w = 1/d^2$	fewer ties; smoother
Radius Distance NN	vote of all points within radius r	number of neighbors varies
Nearest Centroid (NMC)	class of the nearest class mean	very fast; works for compact classes
Prototype / condensed NN	keep only a few representative patterns	reduces storage and time

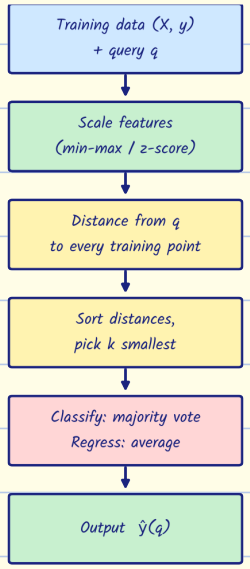


Fig 2.8 : Steps of the KNN algorithm

2.6 K-Nearest Neighbor (KNN) Classifier

Algorithm:

1. Choose k (odd for two classes) and a distance measure; scale the features.
2. For query q compute d(q, xi) to every training pattern xi.
3. Sort the distances and take the k smallest: the k nearest neighbors.
4. **Classify**: assign the **majority class** among them (ties: use smaller k or weights).

Worked example: classify q = (4, 3) using Euclidean distance.

Pattern	Features	Class	d to q = (4,3)	Rank
P1	(1, 1)	A	sqrt(13) = 3.61	6
P2	(2, 2)	A	sqrt(5) = 2.24	2
P3	(2, 5)	A	sqrt(8) = 2.83	3
P4	(5, 3)	B	sqrt(1) = 1.00	1
P5	(7, 3)	B	sqrt(9) = 3.00	4
P6	(5, 0)	B	sqrt(10) = 3.16	5

- **k = 1:** nearest is P4 → class **B**.
- **k = 3:** P4 (B), P2 (A), P3 (A) → A = 2, B = 1 → class **A**.
- **k = 5:** P4 (B), P2 (A), P3 (A), P5 (B), P6 (B) → B = 3, A = 2 → class **B**.
- **Lesson:** the answer **changes with k**, so k is a hyper-parameter that must be tuned.

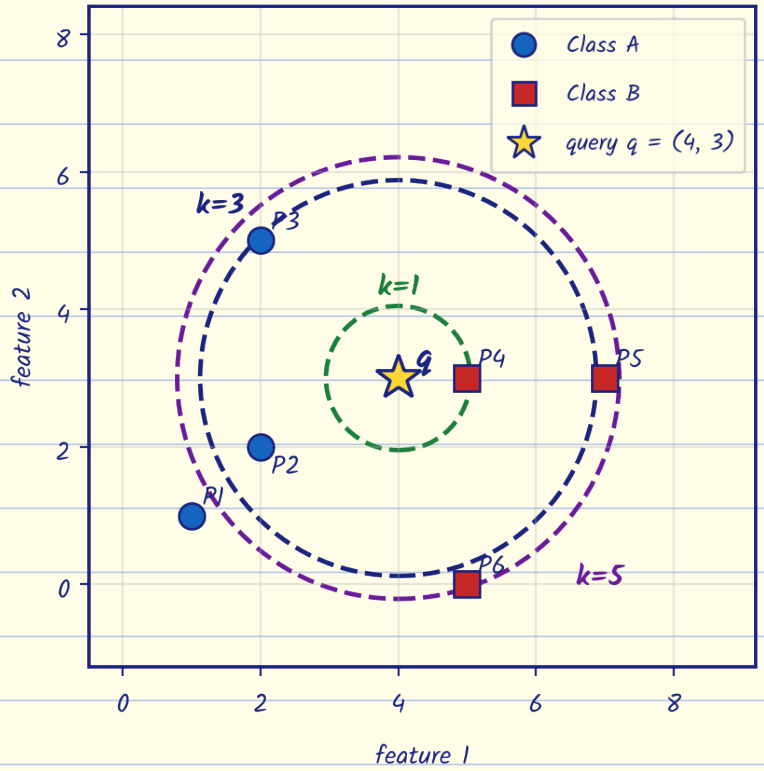


Fig 2.9 : Worked example: query q and its k = 1, 3, 5 neighborhoods

Choosing k

- **Small k** (k = 1): flexible, jagged boundary, follows noise → **over-fitting** (high variance). **Large k:** smooth boundary → **under-fitting** (high bias). k = n always predicts the majority class.
- **Method:** try k = 1, 3, 5, … and pick the k with the lowest **validation / cross-validation error**. Use odd k for 2 classes. Rule of thumb k ≈ sqrt(n).
- Training error at k = 1 is zero (each point is its own neighbor), so it must **never** be used to choose k.

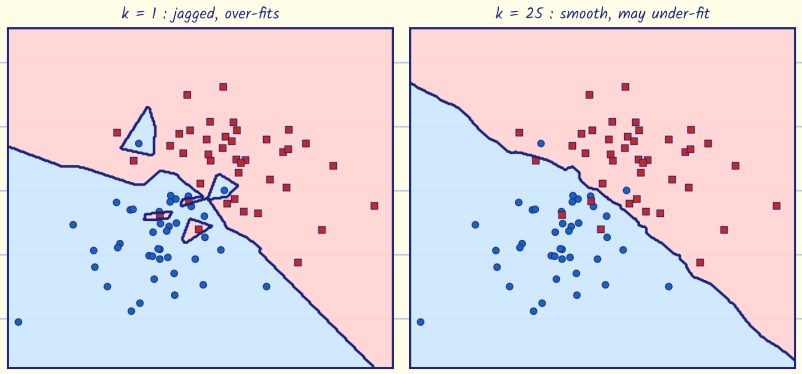


Fig 2.10 : Decision boundary for small and large k

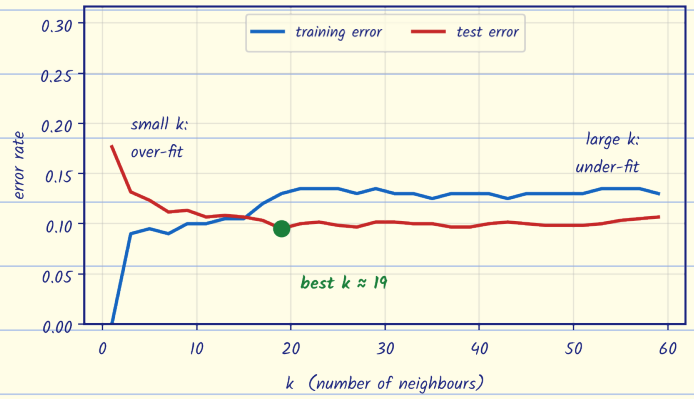


Fig 2.11 : Training and test error against k

Weighted KNN

Give each neighbor a **weight** that falls with distance, $w = 1/d^2$. For q and k = 3: P4 (B): $w = 1/1^2 = 1.000$; P2 (A): $w = 1/2.24^2 = 0.200$; P3 (A): $w = 1/2.83^2 = 0.125$. Votes: B = 1.000, A = 0.325 → class **B**, although plain 3-NN said A.

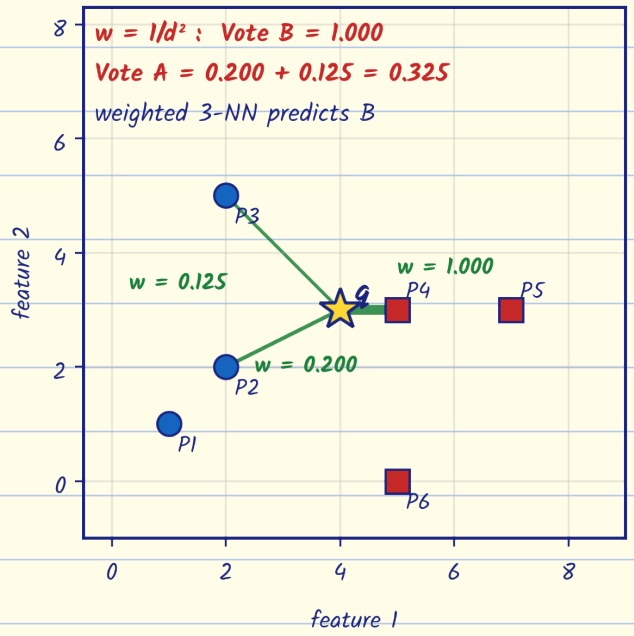


Fig 2.12 : Distance-weighted 3-NN vote

Curse of dimensionality, cost, pros and cons

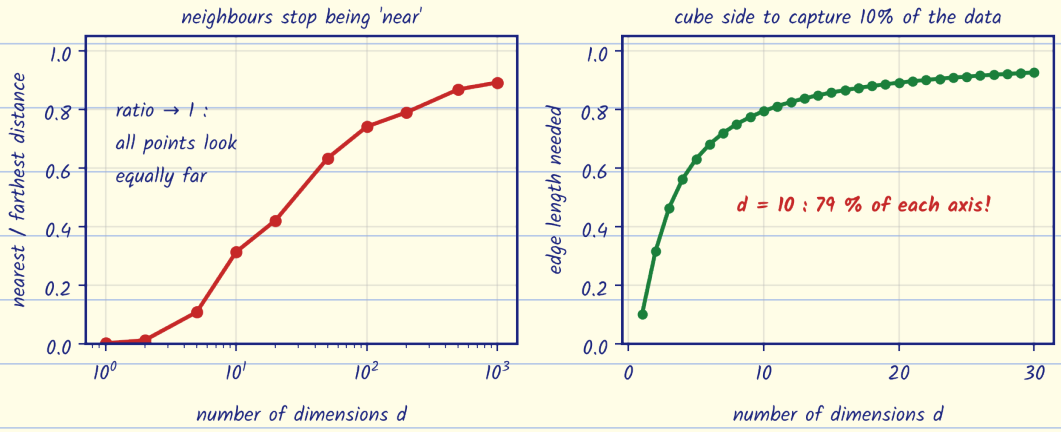


Fig 2.13 : In high dimensions neighbors are no longer 'near'

- **Curse of dimensionality:** as d grows, all points become almost equally far (nearest/farthest → 1) and one needs exponentially more data. Fix: **feature selection, PCA**, more data.
- **Cost:** prediction needs n distance computations of d numbers each → O(n·d) per query. Speed-ups: KD-tree, ball tree, condensed sets, approximate search.

Advantages	Disadvantages
Very simple; no training phase	Slow prediction and large memory (stores all data)
Naturally multi-class	Needs feature scaling; sensitive to irrelevant features
Non-linear boundaries, few assumptions	Poor in high dimensions; sensitive to noise for small k
Works for both classification and regression	Choice of k and distance measure needed

2.7 Radius Distance Nearest Neighbor Algorithm

Instead of a fixed count k, **Radius NN** (fixed-radius NN) uses **all training points within distance r** of the query, and takes the majority class among them.

1. Choose radius r (and distance measure).
2. Find $N(q) = \{ x_i : d(q, x_i) \le r \}$.
3. If N(q) is non-empty: predict the majority (or weighted) class in N(q).
4. If N(q) is empty: output 'outlier / no decision', or use the nearest neighbor, or default to the majority class.

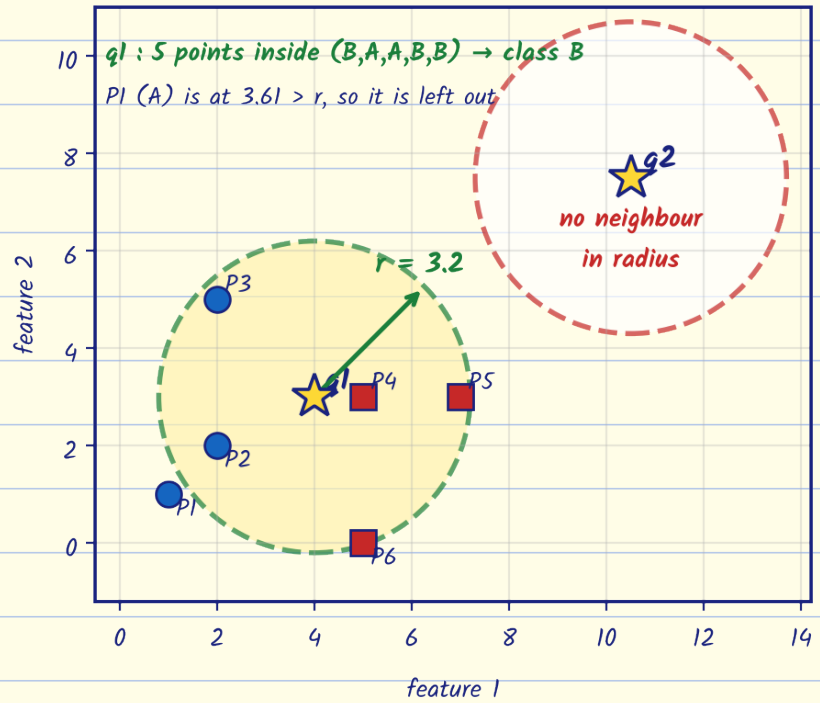


Fig 2.14 : Radius NN with r = 3.2: dense region and empty region

- **Example:** q1 = (4,3), r = 3.2. Distances: P4 1.00, P2 2.24, P3 2.83, P5 3.00, P6 3.16 are inside; P1 (3.61) is outside. Classes B, A, A, B, B → B = 3, A = 2 → **class B**. For q2 far away no point is inside, so it is flagged as an **outlier**.

- **KNN vs Radius NN:** KNN fixes the **number** of neighbors, radius NN fixes the **region**. In dense areas radius NN uses many points, in sparse areas few or none.
- **Choosing r:** too small → many empty neighborhoods; too large → includes other classes (like very large k). Tune r by cross-validation after scaling features.
- **Uses:** outlier / novelty detection, density-based decisions, range queries.

Part III : KNN Regression

2.8 KNN Regression

For a numeric target, the neighbors are found the same way but the prediction is the **average of the neighbors target values** instead of a vote:

Formulas
Plain: $\hat{y}(q) = (1/k) \Sigma\, y_i$ over the k nearest neighbors.
Weighted: $\hat{y}(q) = \Sigma\, w_i \cdot y_i \,/\, \Sigma\, w_i$, $w_i = 1 / d(q, x_i)^2$.

Worked example: training data below, predict y for $q = 4.4$ with $k = 3$.

x	1	2	3	4	5	6	7	8
y	2	3	5	6	6.5	9	10	12

- Distances $|x - 4.4|$: x=4 → 0.4, x=5 → 0.6, x=3 → 1.4 (next is x=6 → 1.6). Neighbors y = 6, 6.5, 5.
- **Plain average:** $(6 + 6.5 + 5)/3 = 17.5/3 = \mathbf{5.83}$.
- **Weighted:** weights $1/0.4^2 = 6.25$, $1/0.6^2 = 2.78$, $1/1.4^2 = 0.51$ → $\hat{y} = (6.25 \cdot 6 + 2.78 \cdot 6.5 + 0.51 \cdot 5) / 9.54 = \mathbf{6.09}$.

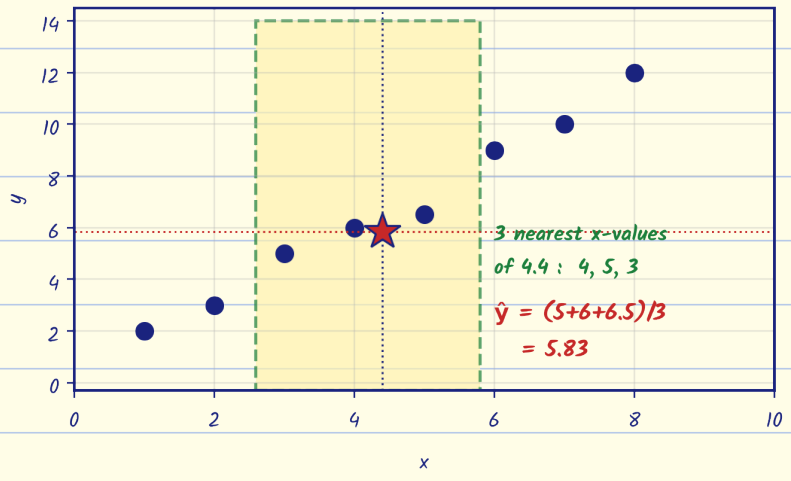


Fig 2.15 : KNN regression with k = 3

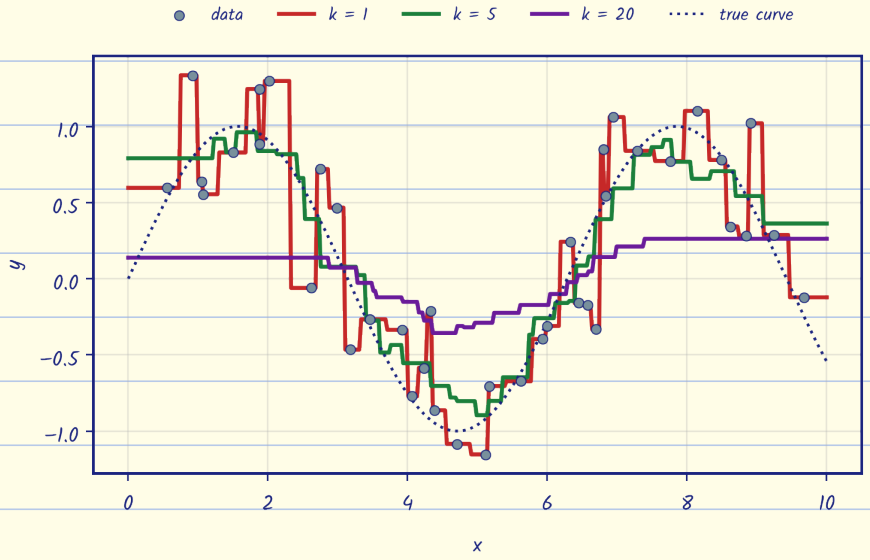


Fig 2.16 : Effect of k on the KNN regression curve

- **Small k**: wiggly curve through the noise (over-fit). **Large k**: flat, over-smooth curve (under-fit). Choose k by cross-validation on RMSE.
- KNN regression is **piecewise constant**, cannot extrapolate beyond the training range (it predicts the average of edge points). Scaling and the curse of dimensionality apply as before.
- Variants: **radius-based** regression (average within r), median instead of mean for outlier robustness.

Part IV : Performance Evaluation

2.9 Performance of Classifiers

Evaluate on **unseen test data**. For a two-class problem the **confusion matrix** counts the four outcomes: TP (positive predicted positive), TN, FP (false alarm), FN (missed positive).

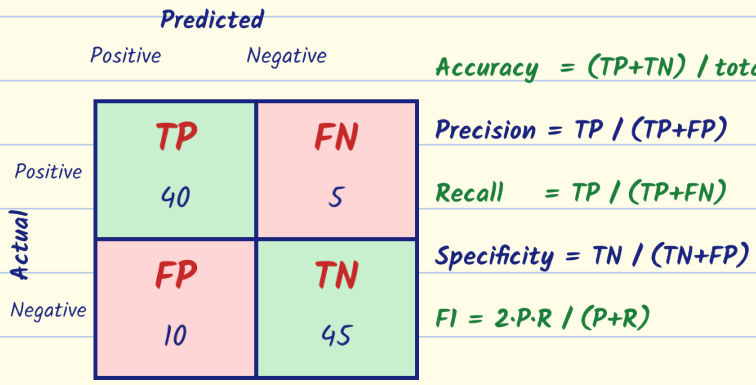


Fig 2.17 : Confusion matrix with TP = 40, FN = 5, FP = 10, TN = 45

Measure	Formula	Value here
Accuracy	(TP + TN) / (TP + TN + FP + FN)	85 / 100 = 0.85
Error rate	1 - accuracy = (FP + FN) / total	0.15
Precision	TP / (TP + FP)	40 / 50 = 0.80
Recall (sensitivity, TPR)	TP / (TP + FN)	40 / 45 = 0.889
Specificity (TNR)	TN / (TN + FP)	45 / 55 = 0.818
F1-score	2·P·R / (P + R) (harmonic mean)	0.842

- **Precision** = of all predicted positives, how many are right. **Recall** = of all real positives, how many were found. Spam filter: high precision. Cancer screening: high **recall**.
- **Imbalanced data:** 99 % negatives → 'always negative' gets 99 % accuracy but recall 0. Use precision, recall, F1.
- **Multi-class:** one confusion matrix of size C × C; per-class precision/recall, then **macro** average (mean of classes) or **micro** average (pool counts).

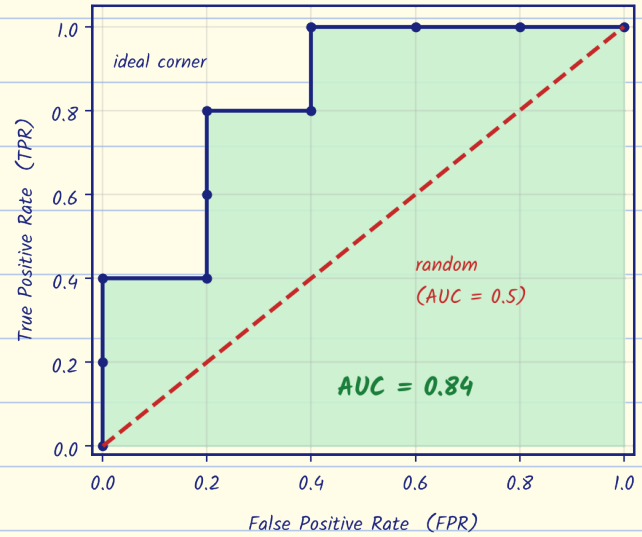


Fig 2.18 : ROC curve: TPR against FPR (AUC = 0.84 for the example)

- **ROC curve:** vary the decision threshold and plot TPR (y) against FPR = FP/(FP + TN) (x). **AUC** = area under it: 1 is perfect, 0.5 is random guessing. (KNN gives a score = fraction of the k neighbors in the class.)
- **Estimating performance:** **hold-out** (e.g. 70 / 30 split), **k-fold cross-validation** (below), **leave-one-out** (k = n), **stratified** splits keep class proportions.

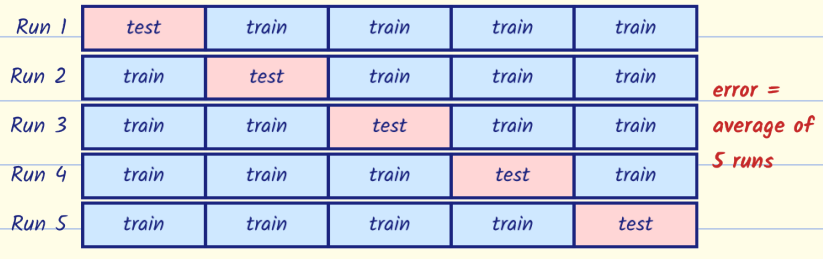


Fig 2.19 : 5-fold cross-validation

2.10 Performance of Regression Algorithms

Compare the predicted ŷ with the actual y. Error e = y - ŷ. Example: actual y = (3, 5, 2.5, 7), predicted ŷ = (2.5, 5, 4, 8) → errors e = (0.5, 0, -1.5, -1).

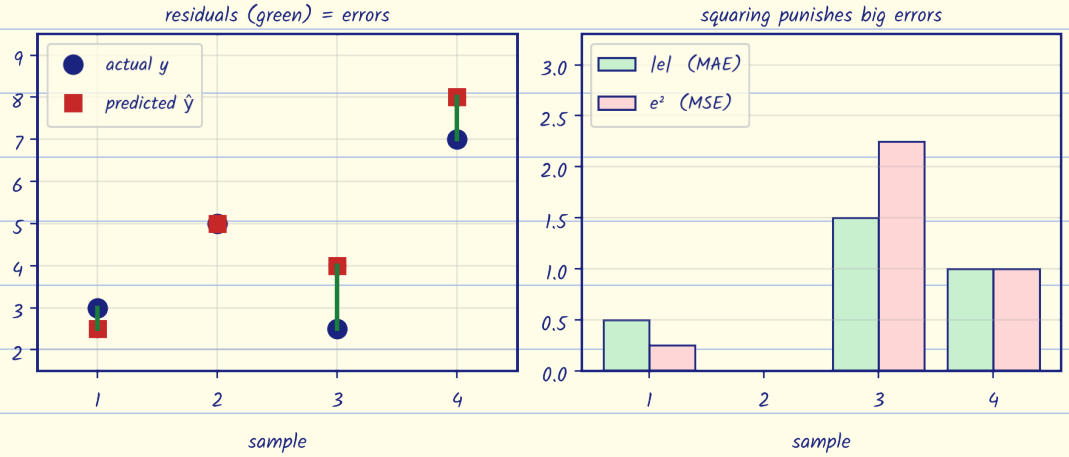


Fig 2.20 : Residuals, absolute error and squared error

Measure	Formula	Value here
MAE	(1/n) Σ |yi - ŷi|	3 / 4 = 0.75
MSE	(1/n) Σ (yi - ŷi)²	3.5 / 4 = 0.875
RMSE	sqrt(MSE) (same unit as y)	0.935
R² (coefficient of determination)	1 - SS_res / SS_tot	1 - 3.5/12.6875 = 0.724

- SS_res = Σ (y - ŷ)² = 0.25 + 0 + 2.25 + 1 = 3.5; mean of y = 4.375; SS_tot = Σ (y - ȳ)² = 12.6875.
- **MAE** treats all errors equally; **MSE/RMSE** punish large errors more (squares) and are more sensitive to outliers.
- **R²**: 1 = perfect, 0 = no better than always predicting the mean, can be negative for a bad model. Here the model explains about 72 % of the variance.

R² = 1 - SS_res / SS_tot (small green gaps vs big red gaps → R² near 1)

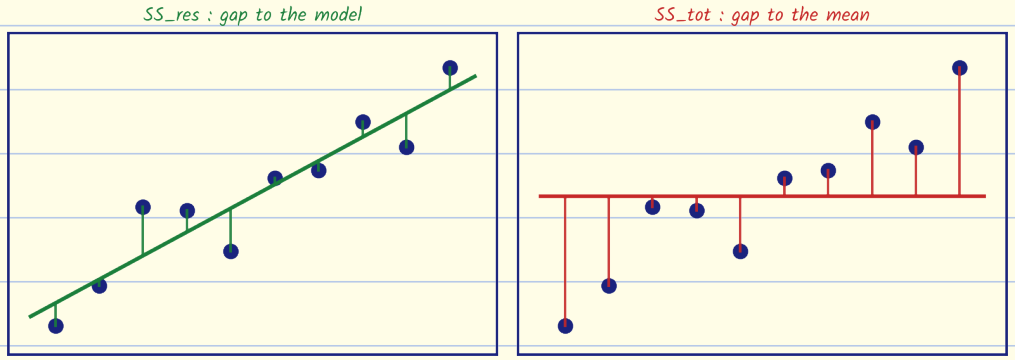


Fig 2.21 : R² compares the model error with the error of the mean

> * **Remember:**
> For a KNN model report errors on a TEST set (or cross-validation) only. On training data the k = 1 model has zero error and looks perfect.

Quick Revision - Unit II

- **Proximity** = similarity (high = alike) or distance (low = alike). KNN is **lazy / instance-based**: no training, all work at query time.
- **Metric**: non-negative, identity, symmetry, triangle inequality.
- **Distances:** Euclidean $\sqrt{\Sigma d^2}$, Manhattan $\Sigma|d|$, Chebyshev $\max|d|$, Minkowski $(\Sigma|d|^p)^{1/p}$, Mahalanobis uses covariance. Order: Cheb ≤ Euc ≤ Man.
- **Non-metric:** cosine, KL (asymmetric), squared Euclidean, fractional Lp, DTW.
- **Binary:** Hamming = b + c; SMC = (a + d)/n; Jaccard = a/(a + b + c); Rao = a/n.
- **KNN:** compute distances → k smallest → majority vote. Small k over-fits, large k under-fits; tune by cross-validation; scale features.
- **Weighted KNN:** $w = 1/d^2$. **Radius NN:** vote of all points within r; empty → outlier.
- **KNN regression:** mean (or weighted mean) of k neighbor targets.
- **Curse of dimensionality:** distances lose meaning in high d; reduce features.
- **Classifier metrics:** accuracy, precision TP/(TP+FP), recall TP/(TP+FN), F1, ROC-AUC, cross-validation.
- **Regression metrics:** MAE, MSE, RMSE, $R^2 = 1 - SS_{res}/SS_{tot}$.

Likely exam questions

1. Define proximity. Explain the properties of a metric and the common distance measures with an example.
2. Compare Euclidean, Manhattan, Minkowski, Chebyshev and Mahalanobis distances.
3. What are non-metric similarity functions? Explain cosine similarity and KL divergence with examples.
4. For binary patterns X = 11010010 and Y = 10011000 compute Hamming, SMC, Jaccard and Rao measures.
5. Explain the K-Nearest Neighbor algorithm. How does the value of k affect the result? How is k chosen?
6. Classify the query point using KNN (k = 1, 3, 5) for the given training table. Show the distance calculation.
7. Explain the radius distance nearest neighbor algorithm and differentiate it from KNN. Describe weighted KNN and KNN regression.
8. Define confusion matrix, accuracy, precision, recall and F1. List the performance measures for regression (MAE, MSE, RMSE, R^2) with an example.